

Modelling and Solving STS Problem with CP, SAT and MIP

Alberto Benni alberto.benni@studio.unibo.it
Lorenzo Rolfini lorenzo.rolfini@studio.unibo.it
Filippo Terzi filippo.terzi2@studio.unibo.it

1 Introduction

We address the Sports Tournament Scheduling (STS) problem in its single round-robin version. An even number n of teams must play over $n - 1$ weeks, with $n/2$ matches per week. In every slot (w, p) one oriented match (i, j) is scheduled, with i at home and j away.

Throughout the report we use the common index sets

$$\text{Teams} = \{1, \dots, n\}, \quad \text{Weeks} = \{1, \dots, n - 1\}, \quad \text{Periods} = \{1, \dots, n/2\},$$

and represent a schedule as two Periods $\times$ Weeks matrices H and A , where $(H_{p,w}, A_{p,w})$ encodes the oriented match played in slot (w, p) . This representation is shared by all models and is used both for solver output and for the official checker.

We consider both a *decision* version (find any feasible schedule) and an *optimization* version that also accounts for fairness. Let $\text{home}_t = |\{(p, w) \mid H_{p,w} = t\}|$ be the number of home games of team t . Since each team plays $n - 1$ games, the ideal split is $(n - 1)/2$; we measure imbalance by

$$\text{dev}_t = \left| \text{home}_t - \frac{n-1}{2} \right| \quad (t \in \text{Teams}),$$

and minimize the aggregate deviation $\sum_t \text{dev}_t$.

At modelling level, all approaches use a core *match-at-slot* layer describing which oriented match is played in each slot (w, p) . In the CP and SAT models we also introduce an explicit *team-at-slot* layer indicating whether a team t occupies slot (w, p) (arrays U in CP and Y in SAT), which is channeled to the match variables.

2 CP Model

Both *decision* (`sts_decision.mzn`) and *optimization* (`sts_opt.mzn`) models use the common index sets and schedule representation introduced in Section 1. In all experiments we consider even values $n \in \{10, 12, 14, 16, 18\}$;

Notation. Iverson brackets $[\cdot]$ denote Boolean $\rightarrow$ integer casts.

2.1 Decision variables

Both CP variants explicitly instantiate the generic match-at-slot and team-at-slot layers of Section 1 as Boolean arrays Z and U . On top of these core variables, we introduce the following additional state.

Decision model

- **C (exact per-period profile):** $C_{t,p} \in \{1, 2\}$ counts the uses of period p by team t and is constrained as in Section 2.3.

Optimization model

- C (as above).
- **W (pair-to-week mapping):** $W_{i,j} \in \text{Weeks}$ for $i < j$ records in which week the match $\{i, j\}$ is played.
- **B (orientation flag):** $B_{i,j} \in \{0, 1\}$ for $i < j$ selects the home/away direction for the pair $\{i, j\}$.
- **Hc , dev , obj (fairness counters and objective):** Hc_t is the number of home matches of team t ; dev_t is the absolute deviation of team t from the ideal split; the objective $obj = \sum_t dev_t$ aggregates the team-wise imbalances and is minimized.

2.2 Objective function

We minimize the total home/away imbalance across teams:

$$\min obj = \sum_{t \in \text{Teams}} dev_t,$$

where dev_t is the absolute difference between team t 's number of home games and the ideal split $(n - 1)/2$. Since each team plays $n - 1$ matches (odd), the smallest per-team imbalance is 1; accordingly, we post the pruning bound

$$n \leq obj \leq 2n$$

2.3 Constraints

Core constraints (both models)

- **One match per slot:** $\forall w, p : \sum_{i < j} Z_{w,p,i,j} = 1$.
- **Each unordered pair once:** $\forall i < j : \sum_{w,p} Z_{w,p,i,j} = 1$.
- **Weekly participation:** $\forall w, t : \sum_p U_{w,p,t} = 1$.
- **Period cap:** $\forall t, p : \sum_w U_{w,p,t} \leq 2$.

Index conventions (unordered pairs) We encode each unordered pair only once and disable meaningless entries:

$$\forall w, p, \forall i \geq j : Z_{w,p,i,j} = 0,$$

$$\text{(optimization)} \quad \forall i \geq j : B_{i,j} = 0, \quad W_{i,j} = 1.$$

These fixed cells are never ranged over (all sums use $i < j$); they simply prevent spurious variables from appearing in the arrays.

Channeling

- $U \leftrightarrow Z$: $\forall w, p, t : U_{w,p,t} = \sum_{j>t} Z_{w,p,t,j} + \sum_{i<t} Z_{w,p,i,t}$.
- $W \leftrightarrow Z$ **(optimization)**: $\forall i < j, \forall w : [W_{i,j} = w] = \sum_p Z_{w,p,i,j}$.
- **Canonical output in the decision model**:
 $\forall w, p : H_{p,w} = \sum_{i<j} Z_{w,p,i,j} i, \quad A_{p,w} = \sum_{i<j} Z_{w,p,i,j} j$.
- **Output (H, A) from placement/orientation (optimization)**:
 $\forall w, p : H_{p,w} = \sum_{i<j} Z_{w,p,i,j} (i B_{i,j} + j(1 - B_{i,j})),$
 $A_{p,w} = \sum_{i<j} Z_{w,p,i,j} (j B_{i,j} + i(1 - B_{i,j})).$

Implied constraints

- **Exact per-team period profile C** : $\forall t, p : C_{t,p} = \sum_w U_{w,p,t}$, and
 $\forall t : \sum_p C_{t,p} = n - 1, \quad \sum_p [C_{t,p} = 1] = 1$.
- **Distinct weeks per team (optimization)**:
 $\forall t : \text{alldifferent}([W_{\min(t,j), \max(t,j)} \mid j \neq t])$ (strong global filtering, as recommended by [3]).
- **Global home-count (optimization)**: $Hc_t = \sum_{j>t} B_{t,j} + \sum_{i<t} (1 - B_{i,t})$
and $\sum_t Hc_t = \frac{n(n-1)}{2}$.

Symmetry breaking

- **Week 1 canonical pairing**: $\forall p : Z_{1,p,2p-1,2p} = \text{true}$.
- **Week 1 orientation (optimization)**: $\forall p : B_{2p-1,2p} = \text{true}$.
- **Single-use period anchor**: $C_{1,1} = 1$.

2.4 Validation

Setup

- **Instances**: $n \in \{8, 10, 12, 14, 16, 18\}$.
- **Solvers**: cp-sat, chuffed, gecode.
- **Time cap**: 300 s per run.

- **Approaches:** symmetry breaking (SB) on/off; implied constraints (IC) on/off.
- **Search (optimization only):** staged search inspired by Trick’s *schedule-then-break* [8]. We separate *structure* from *orientation* and execute phases sequentially:
 - (i) decide the week of each match W first—this exposes the round-robin structure and lets strong globals prune early;
 - (ii) refine in-week placements Z once week domains are tight;
 - (iii) assign home/away B last, as it chiefly affects fairness.
 Heuristics: weighted degree on W to surface difficult week choices; fail-first on the Boolean layers (Z, B) .
- **Result format:** decision tables report runtime in seconds (> 300 if no answer); optimization tables report *time/objective* (objective in **bold** if proven optimal; $> 300/-$ if no solution).

Experimental results: Decision model

n	CP-SAT	Chuffed	Gecode
10	15	> 300	0
12	3	> 300	214
14	11	> 300	> 300
16	> 300	> 300	> 300
18	> 300	> 300	> 300

Table 1: Baseline (SB+IC)

n	CP-SAT	Chuffed	Gecode
10	3	0	74
12	54	5	> 300
14	111	> 300	> 300
16	158	> 300	> 300
18	> 300	> 300	> 300

Table 2: noSB (IC on)

n	CP-SAT	Chuffed	Gecode
10	63	0	0
12	7	0	> 300
14	66	4	> 300
16	151	26	> 300
18	> 300	> 300	> 300

Table 3: noIC (SB on)

n	CP-SAT	Chuffed	Gecode
10	3	0	> 300
12	34	0	> 300
14	147	> 300	> 300
16	178	> 300	> 300
18	> 300	> 300	> 300

Table 4: noIC + noSB

Experimental results: Optimization model

n	CP-SAT	Chuffed	Gecode
10	1/ 10	1/ 10	0/ 10
12	6/ 12	2/ 12	64/ 12
14	23/ 14	61/ 14	> 300/-
16	> 300/-	> 300/-	> 300/-
18	> 300/-	> 300/-	> 300/-

Table 5: Baseline (SB+IC)

n	CP-SAT	Chuffed	Gecode
10	2/ 10	0/ 10	0/ 10
12	7/ 12	2/ 12	> 300/-
14	144/ 14	18/ 14	> 300/-
16	> 300/-	> 300/-	> 300/-
18	> 300/-	> 300/-	> 300/-

Table 6: noSB (IC on)

n	CP-SAT	Chuffed	Gecode
10	1/ 10	1/ 10	> 300/-
12	7/ 12	> 300/-	> 300/-
14	208/ 14	> 300/-	> 300/-
16	> 300/-	> 300/-	> 300/-
18	> 300/-	> 300/-	> 300/-

Table 7: noIC (SB on)

n	CP-SAT	Chuffed	Gecode
10	2/ 10	69/ 10	> 300/-
12	8/ 12	> 300/-	> 300/-
14	197/ 14	> 300/-	> 300/-
16	> 300/-	> 300/-	> 300/-
18	> 300/-	> 300/-	> 300/-

Table 8: noIC + noSB

Discussion

- **Decision.** In the decision version, runs without IC and/or SB often finish sooner than the full model with all extras: this task rewards fast steps over heavy reasoning. IC adds more checks at each step, and SB can remove symmetric solutions that a SAT-based solver might otherwise find quickly.
- **Optimization.** With the staged order $W \rightarrow Z \rightarrow B$, SB and IC help a lot: the “weeks all different” and the period-profile constraints narrow domains before orientations are chosen, and symmetry breaking removes duplicate optimal schedules. Together these effects reduce the work needed to prove optimality.
Because B (home/away) is decided last in the phased search, once a feasible timetable (W, Z) is found the subsequent assignment of B immediately drives the fairness measure to its theoretical minimum; hence the first *complete* solution is typically already optimal.
- **Solver contrast.** `cp-sat` is usually the most reliable. Our formulation has large Boolean parts (Z, U, B) with simple linear sums, which fit SAT-based solvers. This often does better than pure finite-domain propagation when the goal is to get a quick solution.

3 SAT Model

We encode STS in propositional CNF using two families of literals. The solver interface supports Z3 (direct CNF loading) and PySAT backends (DIMACS export with MiniSAT 22/Glucose 3). All runs use a 300s time limit, counted from

CNF generation (when applicable), and results are written under `res/SAT/#.json` in the format required by the project (`time`, `optimal`, `obj`, `sol`).

3.1 Decision variables

Indexing. We follow the common notation of Section 1: *Teams*, *Weeks* and *Periods* index rows, columns and slots of the (H, A) timetable, and slot (p, w) holds exactly one oriented match (i, j) with i at home and j away. The SAT encoding reifies this structure into Boolean literals.

Decision variables (literals)

The generic match-at-slot and team-at-slot layers from Section 1 are reified as Boolean literals: $X_{i,j,w,p}$ and $Y_{t,w,p}$.

These two families are sufficient for the decision version; optimization is handled by adding fairness constraints over the same literals as described in the objective subsection.

Cardinality encodings. We encode ExactlyOne as AtLeastOne+AtMostOne, and we choose the CNF for AtMost(k) adaptively to balance clause/auxiliary-variable overhead and unit-propagation strength. Concretely, for AtMost(1) we use the pairwise encoding on small scopes ($m \leq 8$) because it is aux-free and maximally propagating, whereas we switch to the ladder encoding beyond that to keep the number of clauses linear in m . For $k > 1$ we adopt sequential counters by default and switch to cardinality networks (`cardnetwrk`) when the scope becomes large ($m \geq 15$), trading extra auxiliaries for stronger propagation and better scaling.¹

3.2 Objective function

Optimization in SAT (fairness)

We measure fairness via the maximum home/away deviation per team. Let $T = n - 1$ be the number of games per team in a single round robin and $\text{home}_i = \sum_{j \neq i, w, p} X_{i,j,w,p}$. The *home-away balance* definition [4] asks $|\text{home}_i - \text{away}_i| \leq 1$, which can be simplified to $|\text{home}_i - \frac{T}{2}| \leq \frac{1}{2}$ for all i , i.e., $\text{home}_i \in \{\lfloor T/2 \rfloor, \lceil T/2 \rceil\}$. We generalize this by minimizing a tolerance d :

$$d^* = \min d \quad \text{s.t.} \quad \left\lceil \frac{T-d}{2} \right\rceil \leq \text{home}_i \leq \left\lfloor \frac{T+d}{2} \right\rfloor \quad (\forall i).$$

Parity implies a lower bound $d \geq T \bmod 2$. For fixed d , the admissible region is the product of team-wise intervals and thus *monotone* in d (if feasible for d , then feasible for any $d' > d$). Hence d^* is the smallest d such that the fairness constraints are satisfiable.

¹These thresholds are the ones used in our implementation.

SAT-based solution. We solve by incremental SAT over $d = 1, 2, \dots, T$ (binary search is also possible thanks to monotonicity). For each d , we create team-wise AtLeast/AtMost cardinalities on *home* literals and test satisfiability. With Z3 we maintain a base context and push/pop only the fairness bounds; with PySAT we export the base CNF once and call a SAT solver with the fairness clauses added on top. This realizes a parametric decision procedure for a minimax objective while reusing learned information across d .

3.3 Constraints

Constraints (decision model)

All constraints are posted in CNF over two families of literals: $X_{i,j,w,p}$ and $Y_{t,w,p}$ already explored in section 3.1.

(Global pruning) Week assignment by circle method. We pre-assign to every unordered pair $\{u, v\}$ a unique week w^* by the classical circle construction. Then we forbid all other weeks with unit negatives and enforce an ExactlyOne over the literals $\{X_{u,v,w^*,p}, X_{v,u,w^*,p}\}_p$:

$$\sum_p (X_{u,v,w^*,p} + X_{v,u,w^*,p}) = 1, \quad X_{u,v,w,p} = X_{v,u,w,p} = 0 \quad (\forall w \neq w^*).$$

This removes week permutations while preserving at least one feasible schedule.

Channeling $X \leftrightarrow Y$ (one opponent per week). Let $o(t, w)$ be the unique opponent of t in week w induced by the circle method. For each (t, w, p) :

$$(X_{t,o,w,p} \rightarrow Y_{t,w,p}) \wedge (X_{o,t,w,p} \rightarrow Y_{t,w,p}) \wedge (Y_{t,w,p} \rightarrow (X_{t,o,w,p} \vee X_{o,t,w,p})).$$

These are 2–3 literal clauses, hence cheap and strong.

Exactly one period per team per week. For each (t, w) we post ExactlyOne $\{Y_{t,w,p}\}_p$, which—combined with channeling—means that each team plays exactly once per week.

Exactly one match per slot. For each slot (w, p) we enforce $\sum_t Y_{t,w,p} = 2$ via AtMost2 and its dual (AtLeast2 encoded as AtMost on negated literals). With channeling, two teams in a slot imply exactly one oriented X is true there.

Period load cap. For each team t and period p we impose $\sum_w Y_{t,w,p} \leq 2$ (no team reuses the same period more than twice over the season).

Implied constraints

(i) Because each pair $\{u, v\}$ is week-fixed, the “every team plays with every other team only once” requirement is already implied by the **ExactlyOne** inside week w^* plus the unit negatives for weeks $w \neq w^*$. (ii) Variables with $i = j$ are not created, which implies no self-matches. (iii) “Exactly two teams per slot” together with channeling implies “exactly one match per slot”.

Symmetry breaking

We exploit two levels. (A) *Global*: the circle method fixes the week of each pair, removing all week permutations. (B) *Local (toggle)*: in week 1 the pair $(i, n+1-i)$ is fixed to period i , and match $(1, n)$ is oriented as home for team 1 in $(w, p) = (1, 1)$. This removes residual week/period/orientation symmetries while preserving feasibility.

Why these modelling choices? Symmetry breaking and encoding selection strongly affect SAT performance; we deliberately (i) fix a canonical week assignment via the circle method, (ii) keep local $X \leftrightarrow Y$ channels to obtain short clauses, and (iii) adapt cardinality encodings to scope/threshold. These choices are consistent with the recommendations on the impact of modelling (and symmetry breaking) in SAT problem solving [5].

3.4 Validation

Validation setup

We evaluate each instance under a fixed wall-clock cap of 300 s measured from the start of model/CNF construction to solver termination; the reported time is the total (we also log the build vs. solve split for diagnostics). Execution is pinned to a single CPU core.

We use two back-ends: (i) **Z3**, solving the propositional model in memory; (ii) a **solver-independent** pipeline that exports DIMACS via PySAT and invokes an external SAT solver selected on the command line (e.g., MiniSAT, Glucose).

For decision runs we report wall-clock runtime. For optimization runs we report the time and best objective value found within the cap together with an **optimal** flag.

Experimental results

Tables 9–10 summarize the measured runtimes (seconds) for $n \in \{14, 16, 18\}$ on Z3, MiniSAT 22, and Glucose 3, with/without extra symmetry breaking (“SB”). “-” denotes a timeout per the rules; a valid entry indicates that a model was found (decision) or an optimal d was proven (optimization).

Instance	Z3		MiniSAT 22		Glucose 3	
	SB	no SB	SB	no SB	SB	no SB
$n = 14$	9	2	1	1	3	1
$n = 16$	12	12	117	5	-	109
$n = 18$	53	116	-	-	-	-

Table 9: Decision SAT: runtime (s) and solved status.

Instance	Z3		MiniSAT 22		Glucose 3	
	SB	no SB	SB	no SB	SB	no SB
$n = 14$	6/ 1	4/ 1	1/ 1	1/ 1	1/ 1	5/ 1
$n = 16$	89/ 1	25/ 1	165/ 1	121/ 1	18/ 1	266/ 1
$n = 18$	137/ 1	-	-	-	-	-

Table 10: Optimization SAT: runtime (s) and optimality value.

Observations. (i) For $n = 14$, all solver/back-end combinations easily solve both the decision and the optimization variants. In the decision runs, the additional week-1 symmetry slightly slows down Z3, while its effect on MiniSAT 22 and Glucose 3 is essentially negligible at this size. In optimization, all configurations quickly prove the same optimum $d^* = 1$ within a few seconds.

(ii) For $n = 16$, the decision model is particularly easy for MiniSAT 22 without extra symmetry, whereas Glucose 3 with SB times out and even without SB remains substantially slower. Z3 sits in between. This shows that the additional week-1 symmetry can interact badly with CDCL heuristics for some solvers and that, on this encoding, Glucose 3 is less well-suited than MiniSAT 22 for the pure decision problem.

(iii) In the optimization runs at $n = 16$, all back-ends find the same optimum $d^* = 1$, but runtimes vary widely. Our PySAT pipeline exports the base CNF once and, for each value of d , spawns a fresh CDCL instance to which the fairness clauses are added. The repeated re-initialization and loss of learnt clauses explain why Z3 scales better than most PySAT combinations, even though Glucose 3+SB happens to be very competitive on this specific instance.

(iv) For $n = 18$, only Z3 solves both the decision and the optimization variants. All PySAT runs, as well as the Z3 optimization run without SB, hit the timeout without certifying optimality. This matches the expectation that an in-memory incremental solver benefits more from reusing learnt information and avoiding repeated DIMACS I/O as the instance size grows.

(v) The extra week-1 symmetry has mixed effects. For MiniSAT 22 at $n = 16$ it clearly hurts, while for Glucose 3 on optimization it dramatically helps. This is consistent with the known trade-offs of symmetry breaking in SAT: additional structure can shrink the search space but also disturb activity-based branching heuristics, so the net effect is solver- and instance-dependent.

4 MIP Model

In this section we present the Mixed-Integer Programming (MIP) formulation adopted for the Sports Tournament Scheduling (STS) problem. The model has been implemented in `Pyomo` and solved with different solvers (e.g., CBC, HiGHS, and CPLEX). It follows a two-stage optimization procedure: Stage A aims at finding a feasible period assignment of all matches, while Stage B optimizes the home/away balance for each team given the fixed schedule obtained from Stage A. Both stages reuse the common index sets and timetable representation of Section 1 and are solved sequentially within a fixed overall time budget of 300 seconds.

4.1 Decision variables

Let n denote the number of teams (even), $W = n - 1$ the number of weeks, and $P = n/2$ the number of periods per week, consistently with Section 1. Let $\mathcal{E}_w = \{(i, j) \mid i < j, (i, j) \text{ play in week } w\}$ be the set of matches scheduled in week w according to a round-robin generation.

Stage A. Stage A instantiates the match-at-slot layer of Section 1 via binary variables $x_{ijwp} \in \{0, 1\}$, with $x_{ijwp} = 1$ iff match (i, j) in week w is assigned to period p .

Stage B. Stage B adds the home/away orientation layer via binary variables $s_{ijw} \in \{0, 1\}$, where $s_{ijw} = 1$ when team i plays at home against team j in week w ; the fairness deviations dev_i are exactly those defined in Section 1.

4.2 Objective function

Stage A. Stage A does not include an explicit optimization objective, as it focuses on finding a feasible assignment of all matches to weeks and periods. The solver is therefore stopped as soon as a feasible solution is obtained within the allocated time limit.

Stage B. Given a feasible assignment x^* from Stage A, Stage B minimizes the total imbalance in home/away distribution across all teams:

$$\min \sum_{i=1}^n \text{dev}_i$$

subject to linear inequalities linking dev_i to the number of home matches:

$$\begin{aligned} \sum_{(i,j,w) \in \mathcal{E}} s_{ijw} + \sum_{(j,i,w) \in \mathcal{E}} (1 - s_{jiw}) - \frac{n-1}{2} &\leq \text{dev}_i, \\ \frac{n-1}{2} - \sum_{(i,j,w) \in \mathcal{E}} s_{ijw} - \sum_{(j,i,w) \in \mathcal{E}} (1 - s_{jiw}) &\leq \text{dev}_i. \end{aligned}$$

This ensures that dev_i captures the absolute deviation of each team's home matches from the ideal target.

4.3 Constraints

Stage A (period assignment). Stage A enforces a feasible period assignment by the following linear constraints:

$$\sum_{p=1}^P x_{ijwp} = 1 \quad \forall (i, j, w) \in \mathcal{E}_w \quad (\text{unique period per match}), \quad (1)$$

$$\sum_{(i,j) \in \mathcal{E}_w} x_{ijwp} = 1 \quad \forall w \in \{1, \dots, W\}, \forall p \in \{1, \dots, P\} \quad (\text{unique match per slot}), \quad (2)$$

$$\sum_{\substack{(a,b,w) \in \mathcal{E} \\ a=i \text{ or } b=i}} x_{abwp} \leq 2 \quad \forall i, \forall p \quad (\text{period cap per team}), \quad (3)$$

$$\sum_{\substack{(a,b,w) \in \mathcal{E} \\ a=i \text{ or } b=i}} \sum_{p=1}^P x_{abwp} = 1 \quad \forall i, \forall w \quad (\text{one match per team and week}). \quad (4)$$

To reduce symmetry among equivalent solutions we fix the first week's pairs to predefined period indices,

$$x_{ij1p} = 1 \quad \text{for the } p\text{-th pair in week 1.}$$

Stage B (home/away balance). Stage B uses the deviations dev_i defined in Section 1 and adds a simple symmetry-breaking rule on the orientation of the first match:

$$s_{ij1} = 1 \quad \text{for the first pair } (i, j) \text{ of week 1.}$$

4.4 Validation

The MIP model was implemented in `Pyomo` and executed via the `SolverFactory` interface. The default solver used was `CBC`, with alternative runs using `HiGHS` and `CPLEX` for performance comparison. Note that `CPLEX` is not bundled in the Docker image due to licensing restrictions; only `HiGHS` and `CBC` are available by default.

A total time budget of 300 seconds was imposed for each instance, divided between Stage A and Stage B according to a user-defined split ratio (`splitA` = 0.95 by default).

A greedy warm-start heuristic was employed to initialize feasible values of x_{ijwp} before solving Stage A. Symmetry-breaking constraints were optionally enabled to reduce redundant equivalent solutions. The table reports the time to find the first feasible solution. The objective times coincide, since the solver begins improving the objective immediately after obtaining a feasible solution, with no separate phase.

Experimental results

n	CBC	Highs	CPLEX
10	0/ 10	0/ 10	0/ 10
12	9/ 12	0/ 12	1/ 12
14	9/ 14	62/ 14	0/ 14
16	> 300	113/ 16	2/ 16
18	> 300	> 300	> 300

Table 11: Baseline (SB+WS)

n	CBC	Highs	CPLEX
10	3/ 10	0/ 10	0/ 10
12	0/ 12	0/ 12	0/ 12
14	175/ 14	9/ 14	1/ 14
16	> 300	11/ 16	49/ 16
18	> 300	> 300	> 300

Table 12: noSB (WS on)

n	CBC	Highs	CPLEX
10	0/ 10	0/ 10	0/ 10
12	10/ 12	0/ 12	1/ 12
14	9/ 14	62/ 14	0/ 14
16	> 300	120/ 16	2/ 16
18	> 300	> 300	> 300

Table 13: noWS (SB on)

n	CBC	Highs	CPLEX
10	3/ 10	0/ 10	0/ 10
12	0/ 12	0/ 12	0/ 12
14	192/ 14	9/ 14	1/ 14
16	> 300	11/ 16	49/ 16
18	> 300	> 300	> 300

Table 14: noWS + noSB

5 Conclusions

The CP model benefits from global constraints, implied information on period usage, and explicit symmetry breaking. On the decision variant these features do not always give the fastest answer, but on the optimization variant the staged search on (W, Z, B) and the additional structure are essential, with **cp-sat** emerging as the most robust backend on our instances.

In the SAT setting, the circle method removes week symmetries up front, and fairness is handled by incremental calls over a tolerance parameter. The PySAT+DIMACS pipeline highlights the sensitivity to solver heuristics, while Z3 profits from an in-memory incremental workflow, especially for optimization.

The MIP approach follows a natural two-stage decomposition and can exploit strong off-the-shelf solvers, but scaling to $n = 18$ remains challenging, in particular for CBC.

Overall, the experiments confirm that no single technology dominates across all aspects: CP with a SAT-based backend gives a good performance, SAT

excels when large Boolean structures and incremental solving can be exploited, and MIP is attractive for linear decompositions but is more sensitive to solver choice. More importantly, the results underline that modelling choices (global constraints, symmetry breaking, search order) can be as influential as the choice of solver itself.

6 Authenticity and Author Contribution Statement

This project is the original work of the authors. The models, implementations, and experimental results described in this report were designed and developed by the authors and not copied from other projects or sources. Any external ideas, algorithms, or data that influenced our work are properly cited in the references.

We used the AI system *ChatGPT* as an auxiliary tool only for: (i) helping to draft and refine parts of the written documentation; (ii) assisting in the design and refactoring of the CP runner script; (iii) providing suggestions for debugging when diagnosing coding issues; and (iv) supporting the search for existing material and related work.

Regarding individual contributions: Lorenzo Rolfini was primarily responsible for the Constraint Programming (CP) formulation, the MiniZinc models, and the associated experiments. Alberto Benni was primarily responsible for the SAT encoding, the Z3 and PySAT-based pipelines, and the related experiments. Filippo Terzi was primarily responsible for the Mixed-Integer Programming (MIP) formulation in Pyomo, the two-stage optimisation procedure, and the corresponding experiments. All authors contributed to the overall design of the study and to the writing and revision of the report.

References

- [1] COIN-OR Foundation. *CBC - COIN-OR Branch and Cut Solver*. <https://github.com/coin-or/Cbc>.
- [2] J. van Doornmalen and F. Spieksma. “Integer Programming Models for Round Robin Tournaments”. In: *European Journal of Operational Research* (2023).
- [3] Martin Henz, Tobias Müller, and Sven Thiel. “Global constraints for round robin tournament scheduling”. In: *European Journal of Operational Research* (2004).
- [4] Sigrid Knust and Michael von Thaden. “Balanced home-away assignments”. In: *Discrete Optimization* 3.4 (2006), pp. 354–365. DOI: 10.1016/j.disopt.2006.07.002.

- [5] Ruben Martins. *The Impact of Modelling in SAT Problem Solving*. IST/INESC-ID, Technical University of Lisbon. Discusses modelling choices, symmetry breaking and encodings for Round Robin and related SAT benchmarks.
- [6] Pyomo Project. *Pyomo HowTo Documentation*. <https://pyomo.readthedocs.io/en/stable/howto/index.html>.
- [7] T. Tanaka and R. Miyashiro. “Fair Home-Away Tables in Sports Scheduling”. In: *Journal of Advanced Mechanical Design, Systems, and Manufacturing* 10.3 (2016).
- [8] Michael A. Trick. “A Schedule-Then-Break Approach to Sports Timetabling”. In: *Practice and Theory of Automated Timetabling*. 2000.
- [9] Michael A. Trick. *Sports Scheduling Resources*. <https://mat.tepper.cmu.edu/trick/sched.htm>.
- [10] University of Auckland. *Integer Programming and Sports Scheduling Lecture Notes*. <https://cdn.auckland.ac.nz/>.